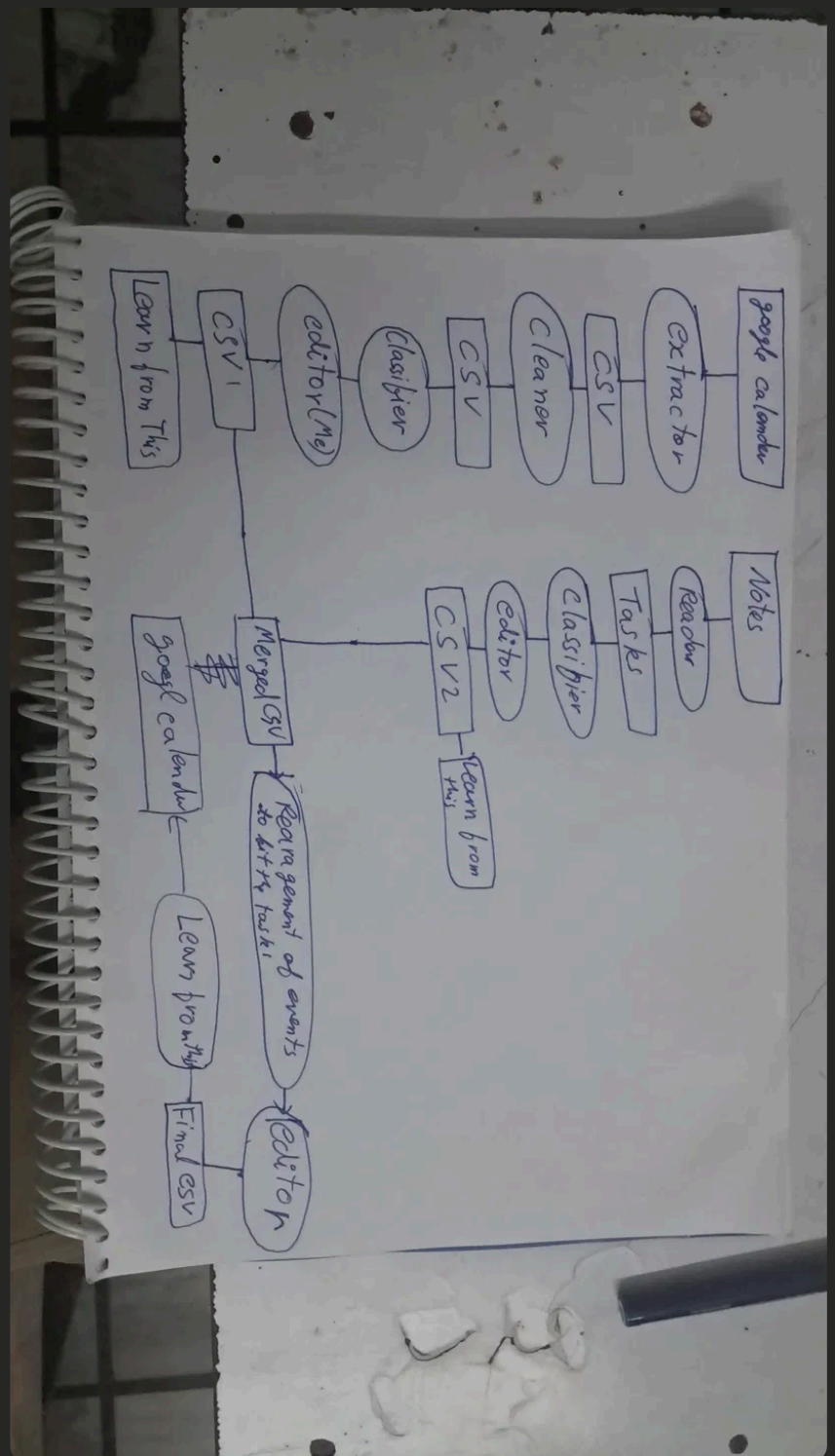


scheduler

07/17/2026 01:10:26

<https://claude.ai/chat/d4bfe5b4-b3ec-464a-86d4-68c7438d8783>

This is my overall project idea, let me know if my direction is correct, Als...
Direction's solid — the real insight is turning your manual corrections...
Yeah locked vs movable is a great idea!, Yeah let's do that, this report...
Good — treating the report as a concept doc, not a spec, is the right call...
Im also planning to use google keep for notes and task keeping, works? ...
Short answer: technically possible, but painful — Google Keep has no...
Okay let's stick to txt file for now
Good — that keeps Phase 0 clean too, since it's just filesystem reads, no...
let's move on to phase 2
Good, Phase 2 doesn't need Calendar working first — you can build and...
* SOP meet 15th aug-5pm * sankalp meet 10th aug-8pm * exam prep 13...
Good sample set — it's got real ambiguity in it, which is exactly what yo...
in some cases like submissions, the deadline/end line is important, but f...
Good catch — that's a real distinction, not a minor detail, and it changes...
give me a sample extraction for my sample notes using this schema
Here's the extraction applied to your 7 lines, using `title, event_type,...
1. we can add another block called "locked | movable" ig that can solve...
Both good instincts — and #2 is actually the more important one, becau...
as you said llm's work is to mainly determine placement flexibility,...
Good — makes sense: "complete report by today" has a hard due-point...
we can make it today as guess
Good — simple, sane default: if a note gives a same-day/next-day range,...
yes
Next step: run this against your 7 notes with the actual LLM you plan to...



Dynamic_ML_Scheduling_Pipeline.PDF

This is my overall project idea, let me know if my direction is correct,

Also tell me steps to get started on this project, dont just code it yourself



Hemang has shared a handwritten flowchart and a detailed project specification f... ▾

- 🕒 Hemang has shared a handwritten flowchart and a detailed project specification for a Dynamic ML Scheduling Pipeline, and he wants me to validate whether his approach is sound and provide actionable next steps—without writing any code.

Let me trace through the flowchart he's drawn. It shows two parallel pipelines: one extracting events from Google Calendar through extraction, cleaning, and classification steps, and another reading tasks from his notes. Both feed into a merged dataset that he then manually rearranges to fit tasks into calendar slots, with the final output syncing back to Google Calendar. The "Learn from this" boxes suggest he's building feedback loops to improve the system over time.

There's a slight discrepancy between what the flowchart shows and the PDF description—the diagram labels the rearrangement step as "editor" doing manual work, whereas the PRD mentions an "Arrangement Model" with hard limits. It looks like the flowchart might be showing his current manual process, while the PDF is describing the intended automated system.

I also notice he's marked himself as "editor(Me)" on the calendar path, which suggests he's planning to be the initial human-in-the-loop for bootstrapping training data. I should give him direct feedback on whether this architecture aligns with his PRD goals and flag that potential gap between the automated arrangement model and the manual editor review.

He's asking for two things: validation of his direction and a concrete roadmap to get started—not code. Given his learning style and the memory notes about his tendency to accumulate projects, I should also consider the timing here. He's prepping for the Expedia hackathon (early-to-mid July, which is basically now) and focusing on placement prep with DSA and SQL. This new scheduling pipeline project is a side initiative, and I need to think about whether to flag the scheduling conflict or just focus on the roadmap he's asking for. ...

- 🕒 Done

Direction's solid — the real insight is turning your manual corrections (Editor 1, Editor 2) into training data instead of work you redo every day. That's the right way to bootstrap a learning system with zero labeled data upfront.

Two things worth resolving before you build:

- The calendar branch's Classifier is ambiguous — is it doing the same 4-tier urgency classification as the notes branch, or something else? Your own Data Model already has the right primitive for this: `Scheduled_Item` has an `Is_Locked` flag. I'd make the calendar-side classifier just decide locked vs. movable (a fixed meeting vs. a routine you're willing to displace), and leave actual priority-tier classification to the notes branch, where it matters more.

- The diagram jumps straight from Merged CSV to "editor" for rearrangement — no box for the actual arrangement logic. That's the highest-risk part per your own Risks section, and the one piece not drawn as a real component. Design it explicitly before writing anything for it.

The design call that matters most: don't let a model output "the schedule" directly. Placing tasks under hard constraints (no overlaps, 24h cap, fixed blocks) is a constraint-satisfaction problem, not something to hand a neural net end-to-end — that's how you'd get exactly the infeasible outputs your Risks section worries about. Split it: ML predicts duration/priority/preferred time; a plain algorithm (greedy interval placement, or OR-Tools if you want a real solver) does the placement, treating locked calendar events as fixed intervals it can't violate.

Steps, in order:

1. Google Calendar API first, before any ML — OAuth working, a script that can list a day's events and create one, against a test calendar. Usually the most annoying non-ML part; get it trivial early.
2. Lock your CSV schemas on paper — exact columns for raw event, parsed task, classified item, scheduled item — before writing any parsing code.
3. Build the dumbest possible end-to-end loop: your own notes format → LLM-prompted extraction + classification (skip custom models for now) → CSV → manual edit → pandas merge → a plain greedy scheduler → manual edit → push to test calendar. Zero learning yet — this is your real MVP, matching your own MVP Recommendation section.
4. Add a JSON/SQLite log of correction history: task → the duration/time the user actually changed it to.
5. After a few weeks of that data, use it — as few-shot examples in your prompts, or as features for a small sklearn classifier. This is where "real" ML earns its place.
6. Only then make the scheduler itself smarter — predictions as tie-breakers, not the whole decision.

One flag unrelated to architecture: this is a meaty build to start the same week as your Expedia hackathon rounds. Steps 1-2 are quick and worth doing now; I'd save the real build for after rounds wrap.

7/5/2026, 12:44:14 PM



Yeah locked vs movable is a great idea!,
Yeah let's do that, this report mainly was my idea of broadly
how the model should work, not the final structure down to the
bones

Also please kind of walk me through it step by step

7/5/2026, 12:51:22 PM



Architected step-by-step implementation roadmap starting with API authentication. ▾

- 🕒 The user is asking for a step-by-step walkthrough on getting started with the project, but they want guidance rather than me just writing the code for them. They're looking for more detailed hand-holding through the initial setup process, building on the earlier list of steps we discussed. The rest of the pipeline flows from there: merge the calendar and notes data using pandas, build the scheduling algorithm as a standalone module that sorts by priority and greedily slots tasks into available time, then review and export back to the test calendar. I'll also track what changes at each manual review step so he can see what decisions were made. This whole thing can be tackled one milestone at a time, starting with getting the Google Calendar API working and authenticated.

🕒 Done

Good — treating the report as a concept doc, not a spec, is the right call. Here's the build order, phase by phase.

Phase 0: Google Calendar plumbing (do this first, it's not ML)

- Go to Google Cloud Console → new project → enable Calendar API → create OAuth credentials (type: Desktop app) → download the JSON.
- Create a second, throwaway Google Calendar just for testing. Get its calendar ID from its settings page. Never point this project at your real calendar until it works.
- Install `google-api-python-client`, `google-auth-httpplib2`, `google-auth-oauthlib`.
- Milestone to hit yourself: a script that authenticates once, lists today's events on the test calendar, and creates one dummy event. Once that works, this phase is done — don't touch it again for weeks.

Phase 1: Lock the schemas on paper

Before any code, write out the exact columns for each CSV stage — you already have

the skeleton in your Data Model section, just make it concrete:

- **Source_CSV**: raw_string, source_type (gcal/notes), is_locked
- **Classified_CSV**: item_id, title, deadline, est_duration, priority_tier
- **Merged_CSV**: everything above combined
- **Final_CSV**: item_id, start_time, end_time, is_locked

Do this in a markdown file. It'll save you from redesigning schemas mid-build.

Phase 2: Notes → structured tasks (LLM prompting, no custom model yet)

- Decide your notes format — pick something dead simple you'll actually type daily (one task per line is fine).
- Write a single prompt template that takes raw notes text and returns JSON: title, deadline, estimated duration. Test it against 10–15 real notes you write yourself, by hand, before wiring it into anything.
- Separately, write a classification prompt: given a task, return one of the 4 tiers. Test this on the same sample set.

Phase 3: Calendar branch — locked vs movable

- For each GCal event, a simple rule (not ML) is probably enough at first: does it recur / was it manually placed by you → locked; otherwise → movable. Save the ML classifier for this branch as a later upgrade, don't build it now.

Phase 4: Editor 1

- No UI needed. Open the classified CSV in a spreadsheet or **csv** module, edit priority tiers by hand, save. The "editor" at MVP stage is just you and a text editor.

Phase 5: Merge

- **pandas.concat** the calendar CSV and the notes CSV into one Merged CSV. This step should be almost trivial — if it's not, your schemas from Phase 1 didn't match up.

Phase 6: Arrangement algorithm — build and test this in isolation

- Don't touch real data yet. Write a function that takes a list of fake tasks (with priority + duration) and fake locked events, and places tasks into the gaps between locked events, highest priority first. This is a greedy interval-scheduling problem, not a model.
- Test it with edge cases you invent: more urgent tasks than time, a locked event smack in the middle of the day, a task longer than any single gap.
- Only after this passes your own stress tests should you feed it real Merged CSV data.

Phase 7: Editor 2

- Same as Editor 1 — manual CSV edit of the arranged schedule.

Phase 8: Log corrections before export

- Before overwriting, diff the pre-edit and post-edit CSVs and append the deltas (task name, predicted duration, actual duration) to a running `corrections_log.csv`. This is the file that makes Phase 9 possible later — start logging from day one even though you won't use it for weeks.

Phase 9: Export

- Push Final CSV rows to the test calendar via the Phase 0 script.

Do these in order — each one should work standalone before you connect it to the next. Skip the real ML (duration learning, sequence preferences) entirely until you have a couple weeks of `corrections_log` data to actually learn from.

7/5/2026, 12:51:54 PM



Im also planning to use google keep for notes and task keeping, works?

Do I need an api key from that too?

7/5/2026, 12:55:31 PM



Short answer: technically possible, but painful — Google Keep has no official public API for personal use.

The catch:

- Google's only sanctioned Keep API is the **Keep API for Google Workspace**, and it's scoped for enterprise data-loss-prevention/backup tools tied to a Google Workspace domain — not built for a personal project like this, and you can't just grab an API key for it the way you can with Calendar.
- The workarounds people use are unofficial libraries like `gkeepapi`, which log in with your actual Google credentials (or an app password) and reverse-engineer Keep's internal sync protocol. That means no OAuth, no official key — just your account credentials sitting in your script. It also breaks unpredictably whenever Google changes something internally, since it's unsupported.

What I'd do instead:

- Skip Keep for this project. Use a plain notes file (a `.txt` or `.md` file you edit locally, or a Google Doc via the official Docs API, which does have proper OAuth) as your "notes" input. Same daily workflow for you, zero fragile dependencies.
- If you specifically want a phone-friendly capture app rather than a text file, Google Tasks has an actual public API with OAuth — that might be a better fit than Keep if you want structured task capture instead of freeform notes.

Given your project's whole premise is parsing unstructured notes, a plain text file is honestly closer to the spirit of what you designed anyway — Keep would just add auth risk for no real benefit here.

7/5/2026, 12:55:37 PM



Okay let's stick to txt file for now

7/5/2026, 12:57:03 PM



Good — that keeps Phase 0 clean too, since it's just filesystem reads, no auth to deal with.

One adjustment to Phase 2 given this: pick a loose convention for the txt file now, even if you don't enforce it strictly — something like one task/thought per line, optionally with a date or  for urgency if you want to hint the parser. You don't need structure, just consistency so your 10-15 test samples in Phase 2 actually resemble what you'll type at 11pm when you're tired. Want me to help sketch a few example lines for that format, or are you moving on to Phase 0 (Calendar API) first?

7/5/2026, 12:57:06 PM



let's move on to phase 2

7/16/2026, 11:39:13 PM



Good, Phase 2 doesn't need Calendar working first — you can build and test this in complete isolation.

Step 1: Write your own sample notes first, by hand

Before any code, sit down and write 10-15 lines the way you'd actually write them at 11pm, tired, not thinking about "the system." Don't sanitize them for the parser. Something like:

```
sankalp meet tmrw need to prep slides  
finish DSA sliding window set before sat  
call mom  
gym  
submit PS-1 report by friday EOD, kinda urgent
```

This matters more than it sounds — if you invent "clean" test notes, your prompt will look like it works and then fail on your real handwriting-brain output. Do this yourself before writing a single line of code.

Step 2: Decide the extraction output shape

Match it to your Data Model from the report: `title`, `extracted_deadline`, `estimated_duration`. Deadline and duration will often be missing from a raw note — decide now that "null when absent" is fine; don't make the LLM guess durations it has no basis for.

Step 3: Write the extraction prompt

One call, one job: raw notes text in, JSON array out. Keep instructions tight — explicit about the output schema, explicit about "return null, don't guess." No classification yet, just extraction.

Step 4: Write the classification prompt separately

Second call, one job: given a task title (+ deadline if present), return one of the 4 tiers. Keep it separate from extraction — easier to debug and iterate independently, and later you can swap just this one out for a real trained classifier without touching extraction at all.

Step 5: Run both against your Step 1 notes and grade it yourself

For each of your 10-15 lines: did it extract the right title? Did it hallucinate a deadline that wasn't there? Is the tier defensible? Keep a running list of failures — this is your real spec for prompt iteration, more useful than anything abstract.

Want to write your sample notes now, or do you want me to draft the two prompt templates first so you have something concrete to test against?

7/16/2026, 11:39:23 PM



- SOP meet 15th aug-5pm
- sankalp meet 10th aug-8pm
- exam prep 13th aug- DSA- 9pm
- weekly revise this saturday
- call babaji(today or tmmrw)
- trip plan
- complete report by today please!!!!

these are my sample notes now waht?

7/16/2026, 11:46:29 PM



Good sample set — it's got real ambiguity in it, which is exactly what you want to stress-test against: relative dates ("today", "this saturday"), a fixed time attached to a date, a note with zero structure ("trip plan"), and one with obvious urgency signaling but no explicit deadline ("please!!!!").

Before running anything, **grade these yourself first** — write down what you'd want extracted for each line. This becomes your answer key.

For example, my own read:

- **SOP meet 15th aug-5pm** → title: "SOP meet", deadline: 2026-08-15 17:00, duration: null
- **sankalp meet 10th aug-8pm** → title: "Sankalp meet", deadline: 2026-08-10 20:00, duration: null
- **exam prep 13th aug- DSA- 9pm** → title: "Exam prep - DSA", deadline: 2026-08-13 21:00, duration: null
- **weekly revise this saturday** → title: "Weekly revise", deadline: next Saturday (needs a reference date — today's date must be passed to the LLM), duration: null
- **call babaji(today or tmmrw)** → title: "Call babaji", deadline: ambiguous — either today or tomorrow, no fixed time. This is a genuinely hard case; decide now

whether the parser should pick one, or flag it as "soft deadline / range" for Editor 1 to resolve.

- **trip plan** → title: "Trip plan", deadline: null, duration: null — no structure at all, purely a routine/someday task.
- **complete report by today please!!!!** → title: "Complete report", deadline: today, duration: null. The **!!!!** is an urgency signal for the *classifier*, not the extractor — don't let the extractor try to encode urgency, that's tier 2's job.

Notice the pattern: your extractor needs **today's actual date** as context to resolve "today", "tmmrw", "this saturday" — that's not optional, it has to go in the prompt every time you run it.

Do you want to write your own answer key for these 7 lines before I draft the extraction prompt, or do you want me to draft the prompt now and you grade its output against your own judgment as you go?

7/16/2026, 11:46:37 PM



in some cases like submissions, the deadline/end line is important, but for meets start time is important, how do i tackle that?

7/16/2026, 11:50:51 PM



Good catch — that's a real distinction, not a minor detail, and it changes what a "deadline" field even means for different task types.

The cleanest way to tackle it: **don't collapse both into one `deadline` field**. Split the concept.

Add a `task_type` (or reuse your priority framing) that distinguishes:

- **Fixed-time events** (meets, calls) → what matters is `start_time`, and often duration is knowable/estimable (a meet is roughly an hour, a call is short).
- **Deadline tasks** (submissions, reports) → what matters is `due_by`, and the actual work can happen any time before that — the model should be free to *place* it, not pin it to the deadline moment.

This maps directly onto your Arrangement phase too:

- Fixed-time events become **locked** (or semi-locked) — like your calendar events, since "SOP meet 5pm" isn't negotiable, the scheduler shouldn't move it.
- Deadline tasks are **movable** — the scheduler picks a slot for the actual work, constrained only by "must finish before `due_by`."

So your extraction schema should really be:

```
title, event_type (fixed_time | deadline_task), anchor_datetime,
estimated_duration
```

Where `anchor_datetime` means *different* things depending on `event_type` — start time for one, due-by for the other. Keep that field name generic in storage, but make the LLM prompt explicit about which one it's populating and why (i.e. include a short

rule in the prompt: "if it's a meeting/call/fixed appointment, anchor_datetime = start time; if it's a task/submission/deliverable, anchor_datetime = due-by time").

This also resolves the ambiguity in your own notes cleanly:

- **SOP meet 15th aug-5pm** → fixed_time, anchor = 5pm Aug 15
- **complete report by today please!!!!** → deadline_task, anchor = end of today
- **call babaji(today or tmmrw)** → fixed_time, but no exact time given — this is the case where you'd want the parser to flag "time unspecified" rather than invent one, and let Editor 1 pin it down.

Want to fold this into the extraction prompt now, or work through a couple more of your sample lines against this new schema first to make sure it holds up?

7/16/2026, 11:50:59 PM



give me a sample extraction for my sample notes using this schema

7/16/2026, 11:53:34 PM



Here's the extraction applied to your 7 lines, using `title, event_type, anchor_datetime, estimated_duration` (assuming today = July 16, 2026 as the reference date the LLM would be given):

```
[
  {
    "title": "SOP meet",
    "event_type": "fixed_time",
    "anchor_datetime": "2026-08-15T17:00",
    "estimated_duration": null
  },
  {
    "title": "Sankalp meet",
    "event_type": "fixed_time",
    "anchor_datetime": "2026-08-10T20:00",
    "estimated_duration": null
  },
  {
    "title": "Exam prep - DSA",
    "event_type": "fixed_time",
    "anchor_datetime": "2026-08-13T21:00",
    "estimated_duration": null
  },
  {
    "title": "Weekly revise",
    "event_type": "fixed_time",
    "anchor_datetime": "2026-07-18T00:00",
    "estimated_duration": null
  }
]
```

```

},
{
  "title": "Call babaji",
  "event_type": "fixed_time",
  "anchor_datetime": null,
  "estimated_duration": null,
  "note": "ambiguous — today or tomorrow, no time given"
},
{
  "title": "Trip plan",
  "event_type": "deadline_task",
  "anchor_datetime": null,
  "estimated_duration": null
},
{
  "title": "Complete report",
  "event_type": "deadline_task",
  "anchor_datetime": "2026-07-16T23:59",
  "estimated_duration": null
}
]

```

A few judgment calls worth flagging, since they're exactly the kind of edge cases you should decide on deliberately rather than let the LLM decide inconsistently:

1. **"Exam prep - DSA - 9pm"** — I called this `fixed_time` because it has an explicit time, but arguably exam prep is *work*, not an appointment. This is genuinely ambiguous. If you want prep sessions to be movable-but-time-boxed, that's actually a **third case**: a deadline task with a *suggested* time rather than a locked one. Worth deciding now rather than letting it slide.
2. **"Weekly revise this saturday"** — no time given, so I anchored it to midnight Saturday as a placeholder date-only value. Whether that's a locked slot or just "sometime Saturday" is the same ambiguity as #1.
3. **"Call babaji"** — I left `anchor_datetime` null and added a `note` field for the ambiguity. You didn't ask for a `note` field in the schema, but I think you need

something like this — a way for the extractor to flag "I couldn't resolve this cleanly" so Editor 1 sees it, rather than silently guessing.

That third point suggests your schema might need one more field: something like `needs_clarification: bool`. Do you want to add that, or would you rather force the LLM to always pick a best guess and let Editor 1 catch mistakes by review instead?

7/16/2026, 11:53:43 PM



1. we can add another block called "locked | movable" ig that can solve most problems
2. all events will be edited by me at first right and llm/ the model should be able to learn that "revise type events are usually placed under this timeslot", also movable/locked block can solve the other concern
3. again edited, also model should learn that tasks with no apparent anchor-time should mostly fall under low urgency, movable events

7/17/2026, 12:04:54 AM



Both good instincts — and #2 is actually the more important one, because it's the difference between "extraction schema" and "how the system learns."

On locked/movable solving #1 and #3: yes, cleanly. Updated schema:

```
title, event_type (fixed_time | deadline_task), anchor_datetime,  
estimated_duration, lock_status (locked | movable)
```

- "SOP meet 5pm" → fixed_time, locked (a real meeting, don't touch it)
- "Exam prep - DSA - 9pm" → fixed_time, **movable** (has a suggested time, but you're free to shift it — this resolves the exam-prep ambiguity from before without a third event_type)
- "Weekly revise" → fixed_time, movable, anchor null-ish/Saturday placeholder — same pattern
- "Call babaji" → fixed_time, movable, anchor null
- "Trip plan" → deadline_task, movable, anchor null
- "Complete report...!!!!" → deadline_task, locked-ish in urgency but not in time — actually this one's interesting: it's not really "locked" in the placement sense, it's just due today. Lock_status should probably describe *placement flexibility*, not

urgency. Keep those separate — urgency is the classifier's job (Phase 2b / Editor 1), lock_status is a placement constraint for the Arranger. Don't let one field try to carry both meanings or you'll end up fighting your own schema later.

On #2 — this is the actual learning mechanism, and it's worth being precise about *where* it lives:

The extractor prompt itself won't "learn" anything — LLM prompts don't have memory between calls unless you feed them something. What you're describing is: *patterns from your corrections_log should get fed back into future prompts.*

Concretely, this means Phase 8 (your corrections log) needs to capture, per task title/keyword: what lock_status and anchor_time you consistently assign to "revise"-type tasks. Then at extraction/classification time, you inject a small "learned preferences" block into the prompt — e.g. "Note: tasks matching pattern 'revise' have historically been scheduled as movable, Saturday afternoon, ~2hr." That's few-shot conditioning, not fine-tuning, and it's the right MVP approach — you don't need a trained model to get this behavior, you need your corrections log queried and summarized into the prompt.

On #3 — "no anchor → low urgency, movable": this one you *can* hardcode as a rule right now, no learning required. It's a sound default, not something that needs data to discover. Put it directly in the classifier prompt as an instruction, and let real correction data override it later only if you find yourself disagreeing with the default often.

So to summarize the split: lock_status + the "no anchor → low urgency" rule are deterministic logic you write today. The "revise → usually Saturday" pattern is the first real instance of your corrections-log-to-prompt feedback loop — worth building deliberately rather than bolting on later, since it's the core mechanism your whole project's "dynamic learning" pitch rests on.

Want to lock this schema and move to writing the actual extraction prompt, or flesh out the corrections-log-to-prompt mechanism a bit more first since it's central to the pitch?

7/17/2026, 12:05:07 AM



as you said llm's work is to mainly determine placement flexibility, classifier will determine urgency, so ig we can put complete report as deadline type, locked

make one more iteration of notes' extracted info

7/17/2026, 12:12:46 AM



Good — makes sense: "complete report by today" has a hard due-point even though the *work* itself could theoretically happen anytime before it, but since it's due *today*, there's effectively no slack left, so locking it is the right practical call. That's actually a nice implicit rule: **deadline_task within N hours of now → locked by default**, movable if there's real runway. Worth noting for later, not needed today.

Here's the updated extraction for all 7 lines:

```
[
  {
    "title": "SOP meet",
    "event_type": "fixed_time",
    "anchor_datetime": "2026-08-15T17:00",
    "estimated_duration": null,
    "lock_status": "locked"
  },
  {
    "title": "Sankalp meet",
    "event_type": "fixed_time",
    "anchor_datetime": "2026-08-10T20:00",
    "estimated_duration": null,
    "lock_status": "locked"
  },
  {
    "title": "Exam prep - DSA",
    "event_type": "fixed_time",
```


1. **SOP and Sankalp meets** — I marked both locked since they're real meetings with a fixed time. Does that match your intent, or do you want *only* things already on your Google Calendar to count as "locked," and notes-derived meets stay movable until you personally confirm them in Editor 1? That's a real design choice about how much the LLM should assume vs. defer to you.
2. **"Call babaji"** — still no `anchor_datetime` since neither "today" nor "tomorrow" is resolvable to a single point. Are you okay with the extractor leaving this null and letting the classifier's "no anchor → low urgency" default handle it, or do you want the extractor to pick the earlier option ("today") as a reasonable guess?

7/17/2026, 12:12:55 AM



we can make it today as guess

7/17/2026, 12:14:40 AM



Good — simple, sane default: if a note gives a same-day/next-day range, take the earlier bound. That's a deterministic rule, no ambiguity for the LLM to wrestle with.

Updated line 5:

```
{  
  "title": "Call babaji",  
  "event_type": "fixed_time",  
  "anchor_datetime": "2026-07-16T23:59",  
  "estimated_duration": null,  
  "lock_status": "movable"  
}
```

Kept it movable and time-unspecified (just end-of-day as the guessed date) rather than locked, since you still have freedom on *when* today to call — just not *whether* it's today.

One thing to pin down before writing the actual prompt: this "pick the earlier option" rule needs to be an explicit instruction in the prompt itself (e.g. "if a note gives a range like 'today or tomorrow,' use the earlier date as anchor_datetime"), not something you hope the LLM infers consistently. Small ambiguous phrasing rules like this are exactly where LLM output gets flaky run-to-run if left implicit.

You've now got a fully decided schema and 7 graded examples — that's actually your complete answer key for Phase 2. Ready for me to draft the actual extraction prompt template now, built to hit this exact schema and these exact edge-case rules?

7/17/2026, 12:14:45 AM



yes

7/17/2026, 12:28:04 AM



The user prompt is empty, so I cannot determine the primary language. However, based on the thinking block being in English, here is the summary: Architected comprehensive date extraction template with JSON formatting.

🕒 I'm drafting a prompt template for extracting dates that includes the schema, extraction rules like today's date injection and the distinction between fixed times and deadline tasks, logic for locked versus movable events, the rule that ambiguous ranges default to earlier dates, and handling for cases without anchors. The output should be formatted as JSON only. I'm considering whether this should be presented as an artifact since it's a reusable technical asset for use in code or API calls.

📎 Extraction prompt template for Phase 2 of the scheduling pipeline

📎 Presenting file(s)...

👌 Done

Next step: run this against your 7 notes with the actual LLM you plan to use (Claude, GPT, whatever your pipeline calls), and check the output against the answer key we just built line-by-line. If it matches, Phase 2 extraction is functionally done and you can move to the classifier prompt (urgency tiers). If something drifts — especially on the fixed_time vs. deadline_task judgment calls — flag it and we'll tighten the prompt before moving on.



extraction_prompt_template

Md



7/17/2026, 12:53:05 AM